

Algorithms

22 August 2026 20:14

An algorithm is a systematic set of instructions combined to solve a complex problem. It is a step-by-finite-step sequence of instructions, each of which has a clear meaning and can be executed in a minimum amount of effort in finite time. In general, an algorithm is a blueprint for writing a program to solve the problem. Once we have a blueprint of the solution, we can easily implement it any high level language like C, C++ and so forth. It solves the problem into the finite number of steps. An algorithm written in a programming language is known as a program. A computer is a machine with no brain or intelligence. Therefore, the computer must be instructed to perform a given task in unambiguous steps. Hence, a programmer must define his problem in the form of an algorithm written in English. Does such an algorithm should have following features:

1. An algorithm should be simple and concise.
2. It should be efficient and effective.
3. It should be free of ambiguity, that is, the logic must be clear.

Similarly, an algorithm must have following characteristics:

- **Input:** It reads the data of the given problem.
- **Output:** The desired result must be produced.
- **Process/definiteness:** Each step or instruction must be unambiguous.
- **Effectiveness:** Each step should be accurate and concise. The desired result should be produced within a finite time.
- **Finiteness:** The number of steps should be finite.

Developing an Algorithm

To develop an algorithm, some steps are suggested:

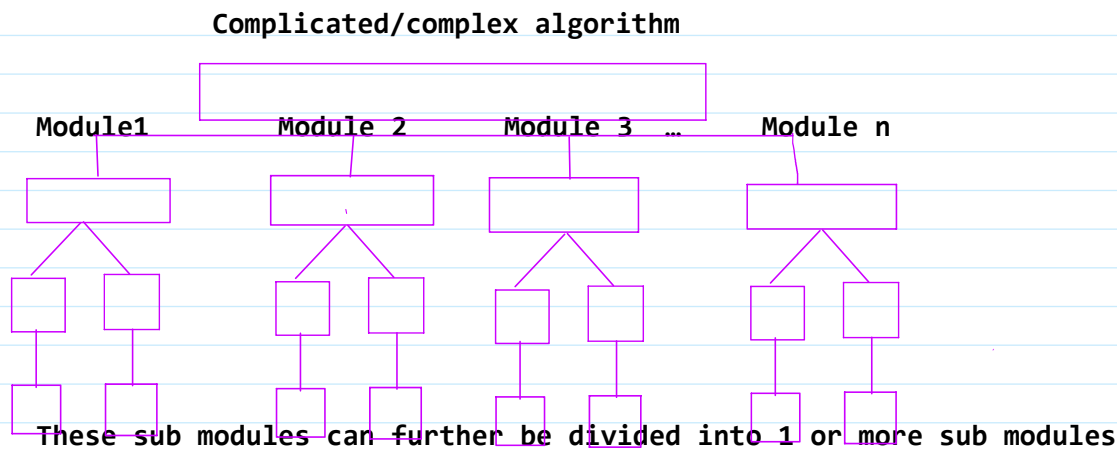
1. Defining or understanding the problem.
2. Identifying the result or output of the problem.
3. Identifying the inputs required by the problem and choosing the best input.
4. Designing the logic from the given inputs to get the desired output.
5. Testing the algorithm for different inputs.
6. Repeating the above steps until it produces the desired result for all the inputs.

Approaches for designing an Algorithm

A complicated algorithm is divided into smaller units, which are called modules. Then these modules are further divided into sub modules. Thus, in this way, a complex algorithm can easily be solved. The process of dividing an algorithm into modules is called modularization. There are two popular approaches for designing an algorithm:

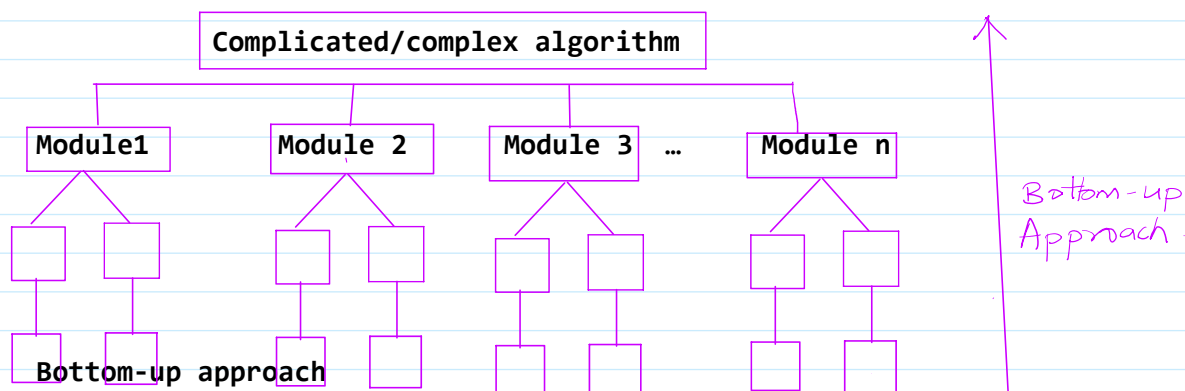
- Top - down approach.
- Bottom-Up Approach.

1. **Top - down approach** - A top-down approach states that complex/complicated problem/algorithm should be divided into a smaller number of one or more modules. These smaller modules are further divided into one or more sub - modules. This process of decomposition is repeated until we achieve the desired output of module complexity. A top - down approach starts from the topmost module and the modules are incremented accordingly until a level is reached where we don't require any more sub - modules. That is the desired level of complexity is achieved.



Top - down approach

2. **Bottom-up Approach** - A bottom up algorithm design approach is the opposite of the top - down approach. In this kind of approach, we first start with designing the basic modules and proceed further towards designing the high level modules. These sub modules are grouped together to form a module of a higher level. Similarly, all high - level modules are grouped to form more high - level modules. Thus this process of combining the sub - modules is repeated until we obtain the desired output of the algorithm.



Analyzing an algorithm

An algorithm can be analyzed by two factors, that is space and time. We aim to develop an algorithm that makes the best use of both these resources. Analyzing an algorithm measures the efficiency of the algorithm. The efficiency of the algorithm is measured in terms of speed and time complexity. The complexity of an algorithm is a function that measures the space and time used by an algorithm in terms of input size.

Time Complexity: The time complexity of an algorithm is the amount of time taken by an algorithm to run the program completely. It is the running time of a

program. The time complexity of an algorithm depends upon the input size. The time complexity is commonly represented by using big O notation. For example, the time complexity of a linear search is $O(n)$.

Space complexity: The space complexity of an algorithm is the amount of memory space required to run the program completely. The space complexity of an algorithm depends upon the input size.

Time complexity is categorized into three types:

1. **Best Case Running Time:** The performance of the algorithm will be best under optimal conditions. For example, the best case for binary search occurs when the desired element is the middle element of the list. Another example can be of salting, that is if the elements are already sorted in a list then algorithm will execute in best time.
2. **Average Case Running Time:** It denotes the behavior of an algorithm when the input is randomly drawn from a given collection or distribution. It is an estimate of the running time for "average" input. It is usually assumed that all inputs of a given size are likely to occur with equal probability.
3. **Worst Case Running Time:** The behavior of the algorithm in this case concern the worst possible case of input instance. The worst case running time of an algorithm is an upper bound on the running time for any input. For example, the worst case for a linear search occurs when the desired element is the last element in the list or the element does not exist in the list.

Q Define the time complexity.

Answer:

Time complexity is a measure which evaluates the count of the operations performed by a given algorithm as a function of the size of the input. It is the approx. Information of the number of steps necessary to execute an algorithm. It is commonly represented with asymptotic notation, that is $O(G)$ Notation, also known as big O notation, where G is the function of the size of the input data.

Time-Space Trade-Off

In computer science time - space trade - off is a way of solving a particular problem either in less time and more memory space or in more time and less memory space. But if we talk in practical terms, designing such an algorithm in which we can save both space and time is a challenging task. So we can use more than one algorithm to solve a problem. One may require less time and other may require less memory space to execute. Therefore, we sacrifice one thing for the other. Hence there exist time-space or time-memory trade-off between algorithms. Say that does this time - space trade - off gives the programmer a rational choice from an informed point of view. So if the time is a big concern for a programmer, then he or she might choose a program which takes less or the minimum time to execute. On the other hand, if space is a prime concern for a program, then in that case he or she might choose a program that takes less memory space to execute, at the cost of more time.

WAA to Search a maximum value from an array X

Saptashwa Ghosh:

1. Start
2. Consider 1st element of the array and store it in a variable max for reference.
3. Move one by one through the elements in the array.
4. Compare each element with the variable max.
5. If variable is less than the array element, the max variable value is replaced by the element.
6. This step is repeated until all the elements have been compared
7. After loop termination we get the maximum value in max variable.
8. End

Soumodip Chakraborty

- 1: take 'a' random value a from the array X and assume it's maximum
- 2: using a loop compare all elements in the array one by one with 'a' variable
- 3: if a value is larger than 'a' variable replace the value of 'a' with element.
- 4: keep repeating until we go through all elements and we'll have the maximum value in variable 'a'

- Variable name must be meaningful, such that the algorithm depicts the logic clearly to the reader.

```
/* Pseudo code for insertion sort */
InsertionSort(A)
Begin
  For J <- 2 to length(A) /* Length(A) means number of elements in A array*/
    Key<-A[j]
    /* Insert A[j] into the sorted sequence A[1]...A[j-1]*/
    i<- J-1
    While i > 0 and A[i] > Key
      A[i+1] <- A[i]
      i <- i-1
    End while
    A[i+1] <- Key
  Endfor
End
```

Analysis of Algorithms

$x = x + 1$

- How much time will be required to execute the statement
- How many times it will? Be processed.

The product of these two outcomes will be the total time taken by the statement.

Another statistics is called frequency count and it may vary from one data sets to another. It is impossible to estimate frequency count unless we have the knowledge about the machine structure, machine cycle time and the speed of the translator.

Asymptotic Analysis

The most obvious way to measure the efficiency of an algorithm would be to run it using a specific set of data and measure how much CPU time and memory space are needed to produce a correct solution. However, this approach seems to work satisfactorily for only one particular set of data, and would be unable to predict how the algorithm would perform using a different set of data. So we need a way to develop a formal technique called asymptotic analysis and will provide a guideline that will allow us to state that for any arbitrary data set, one particular method will probably be the better than another.

Let us consider here three parameters, N , T and S to represent the size of the problem, processing time needed to get the solution, and total memory space required by the solution. For example in searching and sorting a list N would most likely be the number of items in the list. The relationship between N , T and S can be given as:

$$T = f(N)$$

$$S = g(N)$$

The function $f(N)$ is called the time complexity or order of the algorithm and $g(N)$ is called the space complexity of the algorithm.

The notation $f(N) = O[g(N)]$ (read as f of N equals big- O of $g(N)$) has a precise mathematical definition.

Definition: $f(N) = O[g(N)]$ implies there exists k and n_0 such that $[f(N)] < k \cdot g(N)$ for $N > n_0$. $f(N)$ will normally

represents the computing time of some algorithm. The statement 'the computing time of an algorithm is $O[g(N)]$ ' means that the algorithm takes no more than a constant time $g(N)$, where N is the parameter that characterises the input and/or outputs.

The notation $O(1)$ means a computing time which is constant.

$O(n)$ is called linear, $O(n^2)$ is called quadratic, and

$O(n^3)$ is called cubic, $O(2^n)$ is called exponential and

$O(\log_2 n)$ is called order of $\log_2 n$

Computing Functions

n	$\log_2 n$	$n \log_2 n$	n^2	n^3	2^n
1	0	0	1	1	2
2	1	2	4	8	4
4	2	8	16	64	16
8	3	24	64	512	256
16	4	64	256	4096	65536
32	5	160	1024	32768	4294967296

Note that $O(n \log n)$ is better than $O(n^2)$, but not as good as $O(n)$. Similarly, if an algorithm takes $O(\log n)$ it is faster, for sufficiently large n , than if it had taken $O(n)$.

For reasonably large problems, we always want to select algorithms of the lowest order possible.

If algorithm A is $O[f(n)]$ and B is $O[g(n)]$, then algorithm A is lower order than B, if $f(n) < g(n)$ for all n greater than some constant K . For example, $O(n^2)$ is lower order than $O(n^3)$, because $n^2 < n^3$ for all $n > 1$. Similarly, $O(n^3)$ is lower order than $O(2^n)$ since $n^3 < 2^n$ for all $n > 9$.

Thus we would definitely want to select an $O(n^2)$ algorithm to solve a problem, instead of $O(n^3)$ or $O(2^n)$ algorithm, if such an $O(n^2)$ algorithm existed.

